MISE A DISPOSITION DES SERVICES

Ce livrable a pour objectif de fournir une documentation fonctionnelle et descriptive de l'écosystème applicatif conteneurisé. Il sert de référence rapide pour comprendre la valeur et le rôle de chaque service déployé.

Made with GAMMA

LIVRABLE – FICHE DESCRIPTIVE DES SERVICES ET FONCTIONNALITÉS

Document de Restitution – BTS SIO 2025-2026

Auteur : Boutin Samuel

Date : 11/12/2025

Module : Support réseau des accès utilisateurs

Introduction

Pour assurer une lecture claire et une compréhension aisée de l'architecture, les dix applications (WordPress, AppFlowy, Baserow, FreeIPA, n8n, NocoDB, phpMyAdmin/MySQL, PIA, postgres/Adminer, Projector) sont regroupées en pôles fonctionnels distincts :

Pôle Bases de Données & Administration

Les moteurs SGBD (MySQL, PostgreSQL) et leurs interfaces de gestion (phpMyAdmin, Adminer).

Pôle Outils Web & Gestion

Les applications frontales (WordPress, Projector).

Pôle No-Code, Automatisation & Conformité

Les plateformes modernes dépendant de PostgreSQL (n8n, Baserow, NocoDB, PIA).

Pôle AppFlowy & Infrastructure Avancée

La plateforme collaborative et ses micro-services associés (Minio, Nginx).

Pôle Gestion et Monitoring

Pertainer

Chaque fiche de service ci-dessous respecte le formalisme requis, détaillant le **Nom et la Version**, l'**Objectif principal**, les **Fonctionnalités déployées** et surtout les **Interconnexions Applicatives** qui définissent l'architecture réseau logique du projet.

Made with GAMMA

TABLE DES MATIÈRES – ACCÈS RAPIDE

Utilisez cette table des matières pour naviguer rapidement à travers les différentes sections du document.

- Pôle Bases de Données & Administration
- Fonctionnalités et Interconnexions des Bases de Données
- Pôle Outils Web & Gestion
- PROJEQTOR
- Pôle No-Code, Automatisation & Conformité
- APPFLOWY CLOUD
- BASEROW
- N8N
- NOCODB
- PIA (Privacy Impact Assessment)
- Pôle Gestion et Monitoring
- SYNTHÈSE DES RÉSEAUX DOCKER
- Décomposition d'un docker-compose
- Sections Principales et Leur Rôle
- Orchestration des Conteneurs avec Docker Compose
- Nom de variable
- Diagramme Mermaid représentant l'infrastructure

Made with GAMMA

Pôle Bases de Données & Administration

PHPMYADMIN & MYSQL

Description Fonctionnelle

- **Nom** : phpMyAdmin + MySQL
- **Versions** :
  - MySQL: latest à date de création
  - phpMyAdmin: latest à date de création
- **Images Docker** :
  - mysql:latest
  - phpmyadmin/phpmyadmin

Objectif Principal

MySQL : Système de gestion de base de données relationnelle (SGBDR) pour le stockage structuré de données.

phpMyAdmin : Interface web d'administration pour la gestion de bases de données MySQL.

POSTGRESQL & ADMINER

Description Fonctionnelle

- **Nom** : PostgreSQL + Adminer
- **Versions** :
  - PostgreSQL: Custom build (16 avec pgvector)
  - Adminer: latest
- **Images Docker** :
  - Custom build (PostgreSQL) (16 avec pgvector)
  - adminer:latest

Objectif Principal

PostgreSQL : Système de gestion de base de données (SGBD) relationnelle-objet open-source, utilisé comme base de données centrale pour plusieurs applications.

Adminer : Interface web légère pour l'administration de bases de données (alternative à phpMyAdmin, support multi-SGBD).

Made with GAMMA

Fonctionnalités et Interconnexions des Bases de Données

PHPMYADMIN & MYSQL

Fonctionnalités Déployées

MySQL (service 'db') :

- Stockage de données relationnelles
- Gestion de bases de données multiples
- Utilisateurs et permissions

phpMyAdmin :

- Interface graphique pour MySQL
- Exécution de requêtes SQL
- Import/export de bases de données
- Gestion des utilisateurs et privilèges
- Visualisation de la structure des tables

Interconnexions Applicatives

- **phpMyAdmin vers MySQL** :
  - Type : Connexion TCP au service 'db'
  - Port : 3306
  - Nature : Communication SQL via protocole MySQL
  - Configuration : PMA_HOST=db
- **Réseaux multiples** :
  - 'wordpress' : Permet à WordPress et ProjeQtor d'accéder à MySQL
  - 'wordpress3' : Permet à WordPress_2 d'accéder à MySQL

POSTGRESQL & ADMINER

Fonctionnalités Déployées

PostgreSQL (service 'postgresdb') :

- Gestion de bases de données multiples
- Support avancé des types de données
- Transactions ACID
- Réplication et haute disponibilité
- Extensibilité (extensions PostgreSQL)

Adminer :

- Interface web unifiée
- Support multiple SGBD (PostgreSQL, MySQL, SQLite, etc.)
- Éditeur SQL avec auto-complétion
- Import/export de données
- Gestion des schémas et tables

Interconnexions Applicatives

- **Services utilisant PostgreSQL** :
  - **Baserow** : Stockage des bases de données et métadonnées
  - **n8n** : Stockage des workflows et historique
  - **NocoDB** : Stockage des configurations
  - **PIA** : Stockage des analyses RGPD
  - **AppFlowy** : Stockage des données utilisateurs (connexion externe configurée)
- **Adminer vers PostgreSQL** :
  - Type : Connexion directe TCP au service 'postgresdb'
  - Port : 5432
  - Nature : Interface d'administration web via protocole PostgreSQL
- **Volume persistant** : postgresdb pour la persistance des données
- **Réseau 'pia'** : Tous les services PostgreSQL-dépendants y sont connectés

Made with GAMMA

Pôle Outils Web & Gestion

WORDPRESS (2 instances)

Description Fonctionnelle

- **Nom** : WordPress (instances multiples)
- **Version** : latest à date de création
- **Image Docker** : wordpress:latest

Objectif Principal

Système de gestion de contenu (CMS) permettant la création et l'administration de sites web dynamiques. Deux instances WordPress indépendantes sont déployées pour gérer des sites web distincts, chacune avec sa propre base de données et son propre réseau d'isolation.

Fonctionnalités Déployées

- Création et gestion de pages et articles
- Interface d'administration complète
- Système de thèmes et plugins
- Gestion multi-utilisateurs
- Publication de contenu multimédia

Architecture Multi-Instances

Instance 1 - wordpress_bridged :

- **Container** : wordpress_bridged
- **Port hôte** : 80:80 (HTTP standard)
- **Base de données** : wdp1
- **Volume** : /home/sam/Work/wordpress/wordpress:/var/www/html (bind mount absolu)
- **Réseau** : wordpress (externe)
- **Politique de redémarrage** : unless-stopped

Instance 2 - wordpress_2 :

- **Container** : wordpress_2
- **Port hôte** : 82:80 (HTTP sur port alternatif)
- **Base de données** : wdp2
- **Volume** : /wordpress_2:/var/www/html (bind mount relatif)
- **Réseau** : wordpress3
- **Politique de redémarrage** : unless-stopped

Interconnexions Applicatives

Instance 1 (wordpress_bridged) :

- **Connexion à MySQL (service 'db') :**
  - Type : Connexion base de données MySQL via protocole TCP
  - Port : 3306
  - Configuration : WORDPRESS_DB_HOST=db:3306, WORDPRESS_DB_USER=wdpuser, WORDPRESS_DB_PASSWORD=wddppassword, WORDPRESS_DB_NAME=wdp1
  - Nature : Stockage des données du site (articles, pages, utilisateurs, configurations)
- **Réseau 'wordpress' (externe)** : Partage le réseau avec MySQL et phpMyAdmin
- **Volume bind mount** : Chemin absolu pour accès direct aux fichiers

Instance 2 (wordpress_2) :

- **Connexion à MySQL (service 'db') :**
  - Type : Connexion base de données MySQL via protocole TCP
  - Port : 3306
  - Configuration : WORDPRESS_DB_HOST=db:3306, WORDPRESS_DB_USER=wdpuser (même utilisateur), WORDPRESS_DB_PASSWORD=wddppassword, WORDPRESS_DB_NAME=wdp2 (base séparée)
  - Nature : Stockage des données du second site (isolation des contenus)
- **Réseau 'wordpress3' :** Réseau séparé connecté au même service MySQL (isolation réseau)
- **Volume bind mount :** Chemin relatif pour portabilité

Séparation et Isolation

- **Bases de données distinctes** (wdp1 vs wdp2) : Garantit l'isolation complète des données entre les deux sites
- **Réseaux Docker séparés** (wordpress vs wordpress3) : Isolation réseau tout en accédant au même MySQL via double attachment du service 'db'
- **Ports différents** (80 vs 82) : Permet l'accès simultané aux deux sites depuis l'hôte

Made with GAMMA

PROJEQTOR

Description Fonctionnelle

- **Nom :** ProjeQtor
- **Version :** 12.2.3-1
- **Image Docker :** mraber/projeqtor:12.2.3-1

Objectif Principal

Logiciel de gestion de projet intégré (ERP/PMO) open-source, permettant la planification, le suivi et le pilotage de projets avec gestion complète du cycle de vie.

Fonctionnalités Déployées

- Gestion de projets et planning (Gantt)
- Suivi des tickets et incidents
- Gestion des risques et exigences
- Suivi du temps et des coûts
- Gestion documentaire
- Tableaux de bord et reporting
- Gestion multi-projets et portefeuilleInterconnexions de ProjeQtor et Pôle No-Code

Interconnexions Applicatives

- **Connexion à MySQL (service 'db') :**
  - Type : Connexion base de données MySQL en TCP
  - Configuration : PJJ_DB_HOST="db:3306"
  - Base de données : projeqtor
  - Nature : Stockage de toutes les données projets, tickets, ressources, temps
- **Réseau 'wordpress' :** Partage le réseau avec MySQL et phpMyAdmin
- **Volumes persistants :**
  - prj_attachments : Pièces jointes des tickets/projets
  - prj_documents : Documents du projet
  - prj_logs : Logs applicatifs

Made with GAMMA

Pôle No-Code, Automatisation & Conformité

APPFLOWY CLOUD

Plateforme collaborative open-source alternative à Notion, permettant la gestion de projets, la prise de notes et l'organisation du travail en équipe avec synchronisation cloud et fonctionnalités d'IA.

BASEROW

Base de données collaborative no-code alternative à Notion, Google Sheets, NocoDB, Airtable et ClickUp, permettant de créer et gérer des bases de données relationnelles via une interface web intuitive.

N8N

Plateforme d'automatisation workflow (alternative à Zapier) permettant de connecter différentes applications et services pour automatiser des processus métier.

NOCODB

Transforme n'importe quelle base de données en interface type "smart spreadsheet", offrant une alternative open-source à Notion, Google Sheets, BaseRow, Airtable et ClickUp avec génération automatique d'API REST.

PIA (Privacy Impact Assessment)

Outil officiel de la CNIL pour réaliser des analyses d'impact relatives à la protection des données conformément au RGPD.

Made with GAMMA

APPFLOWY CLOUD

Description Fonctionnelle

- **Nom** : AppFlowy Cloud (écosystème complet)
- **Version** : latest à date de création pour tous les services
- **Images Docker** :
  - appflowyinc/appflowy_cloud
  - appflowyinc/gotrue
  - appflowyinc/admin_frontend
  - appflowyinc/appflowy_ai
  - appflowyinc/appflowy_worker
  - appflowyinc/appflowy_web
  - nginx
  - minio/minio
  - redis

Fonctionnalités Déployées

- Gestion de documents collaboratifs
- Authentification utilisateur (OAuth Google, GitHub, Discord, SAML)
- Stockage de fichiers (S3/MinIO)
- Intelligence artificielle intégrée (GPT-4)
- Interface web et administration
- Traitement asynchrone des tâches (worker)
- Notifications par email (SMTP)

Interconnexions Applicatives

Service nginx :

- Type : Reverse proxy HTTP/HTTPS
- Ports : 85:80 (HTTP), 7443:443 (HTTPS)
- Rôle : Point d'entrée unique pour tous les services AppFlowy
- Volume partagé : Configuration nginx et certificats SSL

Service minio :

- Type : Stockage d'objets compatible S3
- Port interne : 9000 (API), 9001 (console)
- Rôle : Stockage de fichiers et documents
- Interconnexion : Utilisé par appflowy_cloud, appflowy_ai et appflowy_worker via variables d'environnement S3

Service redis :

- Type : Base de données en mémoire / Cache
- Port : 6379
- Rôle : Cache et gestion des files de messages
- Interconnexion : Utilisé par appflowy_cloud et appflowy_worker (APPFLOWY_REDIS_URI)

Service gotrue :

- Type : Service d'authentification
- Port : 9999
- Rôle : Gestion de l'authentification et des sessions utilisateurs
- Interconnexion :
  - Connexion PostgreSQL externe via DATABASE_URL
  - API REST utilisée par tous les autres services AppFlowy
  - Healthcheck HTTP pour vérification de disponibilité

Service appflowy_cloud :

- Type : Backend principal
- Port : 8001
- Rôle : API centrale de l'application
- Interconnexions :
  - Dépendance : gotrue (service_healthy)
  - Connexion PostgreSQL via APPFLOWY_DATABASE_URL
  - Connexion Redis via APPFLOWY_REDIS_URI
  - Communication avec MinIO pour le stockage
  - Communication avec service AI (AI_SERVER_HOST:AI_SERVER_PORT)

Service ai :

- Type : Service d'intelligence artificielle
- Port : 5001
- Rôle : Traitement des requêtes IA (GPT-4)
- Interconnexions :
  - API OpenAI externe
  - Connexion PostgreSQL (AI_DATABASE_URL)
  - Connexion Redis (AI_REDIS_URL)
  - Accès MinIO pour traitement de fichiers
  - Dépendance : appflowy_cloud (service_healthy)

Service admin_frontend :

- Type : Interface d'administration web
- Rôle : Console d'administration pour gérer les utilisateurs et paramètres
- Interconnexions :
  - Dépendances : gotrue et appflowy_cloud (service_healthy)
  - API REST vers gotrue et appflowy_cloud

Service appflowy_worker :

- Type : Worker de traitement asynchrone
- Rôle : Traitement des tâches en arrière-plan (imports, exports, notifications)
- Interconnexions :
  - Redis pour la file de messages (APPFLOWY_WORKER_REDIS_URL)
  - PostgreSQL pour accès aux données
  - MinIO pour manipulation de fichiers
  - SMTP pour envoi d'emails

Service appflowy_web :

- Type : Interface web frontend
- Rôle : Application web client
- Interconnexions :
  - WebSocket vers appflowy_cloud (APPFLOWY_WS_BASE_URL)
  - API REST vers appflowy_cloud et gotrue

Réseau commun 'pia' : Tous les services communiquent via ce réseau Docker bridge isolé

Made with GAMMA

BASEROW

Description Fonctionnelle

- **Nom** : Baserow
- **Version** : 1.35.3
- **Image Docker** : baserow/baserow:1.35.3

Fonctionnalités Déployées

- Création de tables et bases de données visuelles
- Gestion de différents types de champs (texte, nombre, date, relation, etc.)
- Vues personnalisables (grille, galerie, formulaire)
- API REST automatique
- Collaboration multi-utilisateurs
- Import/export de données

Made with GAMMA

N8N

Description Fonctionnelle

- **Nom** : n8n
- **Version** : latest à date de création
- **Image Docker** : docker.n8n.io/n8nio/n8n

Fonctionnalités Déployées

- Éditeur visuel de workflows
- Connecteurs vers plus de 400 services (API, bases de données, etc.)
- Exécution programmée de workflows (cron)
- Webhooks pour déclenchement externe
- Gestion des credentials sécurisée
- Interface web d'administration

Interconnexions Applicatives

- **Connexion à PostgreSQL (service 'postgresdb')** :
  - Type : Connexion base de données PostgreSQL
  - Configuration : DB_POSTGRESDB_HOST, DB_POSTGRESDB_PORT (5432)
  - Nature : Stockage des workflows, exécutions, credentials
- **Réseau 'pia'** : Accès au service postgresdb et possibilité de communiquer avec d'autres services du réseau
- **Volumes persistants** :
  - n8n_data : Configuration et données de l'application
  - n8n_files : Fichiers temporaires et uploads

Made with GAMMA

NOCODB

Description Fonctionnelle

- **Nom :** NocoDB
- **Version :** latest à date de création
- **Image Docker :** nocodb/nocodb:latest

Fonctionnalités Déployées

- Interface type tableau pour bases de données
- Création de vues (Grid, Gallery, Form, Kanban)
- Relations entre tables
- API REST et GraphQL auto-générées
- Gestion des permissions utilisateurs
- Intégrations webhooks

Interconnexions Applicatives

- **Connexion à PostgreSQL (service 'postgresdb')** :
  - Type : Connexion base de données PostgreSQL via URI
  - Configuration : NC_DB="pg://postgresdb:5432?u=piauser&p=Password1&d=nocodb"
  - Nature : Stockage des métadonnées et configuration de NocoDB
- **Réseau 'pia'** : Communication avec postgresdb
- **Volume persistant** : nc_data pour le stockage applicatif

Made with GAMMA

PIA (Privacy Impact Assessment)

Description Fonctionnelle

- **Nom :** PIA CNIL
- **Version :** Custom build (Angular Cli 11.2.0 / nodejs 16-alpine / nginx 1.21-alpine)
- **Images Docker :**
  - cnil-pia-back (backend Ruby on Rails version 2.7.4) version 3.0.0
  - cnil-pia-front (frontend) version 3.0.2

Fonctionnalités Déployées

- Création et gestion de PIA
- Évaluation des risques RGPD
- Génération de rapports
- Gestion multi-projets
- Interface d'édition collaborative

Interconnexions Applicatives

- **PIA-back vers PostgreSQL (service 'postgresdb') :**
  - Type : Connexion base de données PostgreSQL
  - Configuration : DATABASE_HOST=postgresdb
  - Port : 5432
  - Nature : Stockage de toutes les analyses PIA et données utilisateurs
- **PIA-front vers PIA-back :**
  - Type : Communication API REST
  - Port : 3000
  - Nature : L'interface frontend communique avec le backend via HTTP
- **Réseau 'pia' :** Les deux services communiquent sur ce réseau isolé avec accès à postgresdb

Made with GAMMA

Pôle Gestion et Monitoring

PORTAINER

Description Fonctionnelle

- **Nom** : Portainer Community Edition
- **Version** : latest à date de création
- **Image Docker** : portainer/portainer-ce:latest

Objectif Principal

Interface web de gestion et d'administration pour environnements Docker, permettant la supervision, le déploiement et la gestion des conteneurs, images, volumes et réseaux.

Fonctionnalités Déployées

- Visualisation de tous les conteneurs Docker
- Gestion des stacks Docker Compose
- Monitoring des ressources (CPU, RAM, réseau)
- Gestion des images et volumes
- Accès aux logs des conteneurs
- Gestion des réseaux Docker
- Interface de terminal pour conteneurs

Interconnexions Applicatives

- **Accès au Docker Engine :**
  - Type : Socket Unix
  - Volume : /var/run/docker.sock:/var/run/docker.sock
  - Nature : Communication directe avec le daemon Docker pour contrôle total de l'hôte
  - Permission : Accès root sur Docker (contrôle complet)
- **Aucun réseau Docker personnalisé** : Utilise le réseau bridge par défaut pour gérer tous les autres réseaux
- **Volume persistant** : portainer_data pour la configuration et les données

Made with GAMMA

SYNTHÈSE DES RÉSEAUX DOCKER

Réseau 'pia'

Services connectés : PostgreSQL, Adminer, Baserow, n8n, NocoDB, PIA-back, PIA-front, AppFlowy (tous services)

Rôle : Réseau principal pour les applications utilisant PostgreSQL

Configuration : Bridge avec liaison à 127.0.0.1

Réseau 'wordpress'

Services connectés : MySQL (db), phpMyAdmin, WordPress, ProjeQtor

Rôle : Réseau pour les applications utilisant MySQL

Configuration : Bridge avec liaison à 127.0.0.1

Réseau 'wordpress3'

Services connectés : MySQL (db), WordPress

Rôle : Réseau pour les applications utilisant MySQL

Configuration : Bridge avec liaison à 127.0.0.1

Réseau 'portainer_default'

Services connectés : Portainer (l'interface d'administration graphique).

Rôle : Réseau d'administration. Il permet l'accès à l'interface de gestion de Portainer.

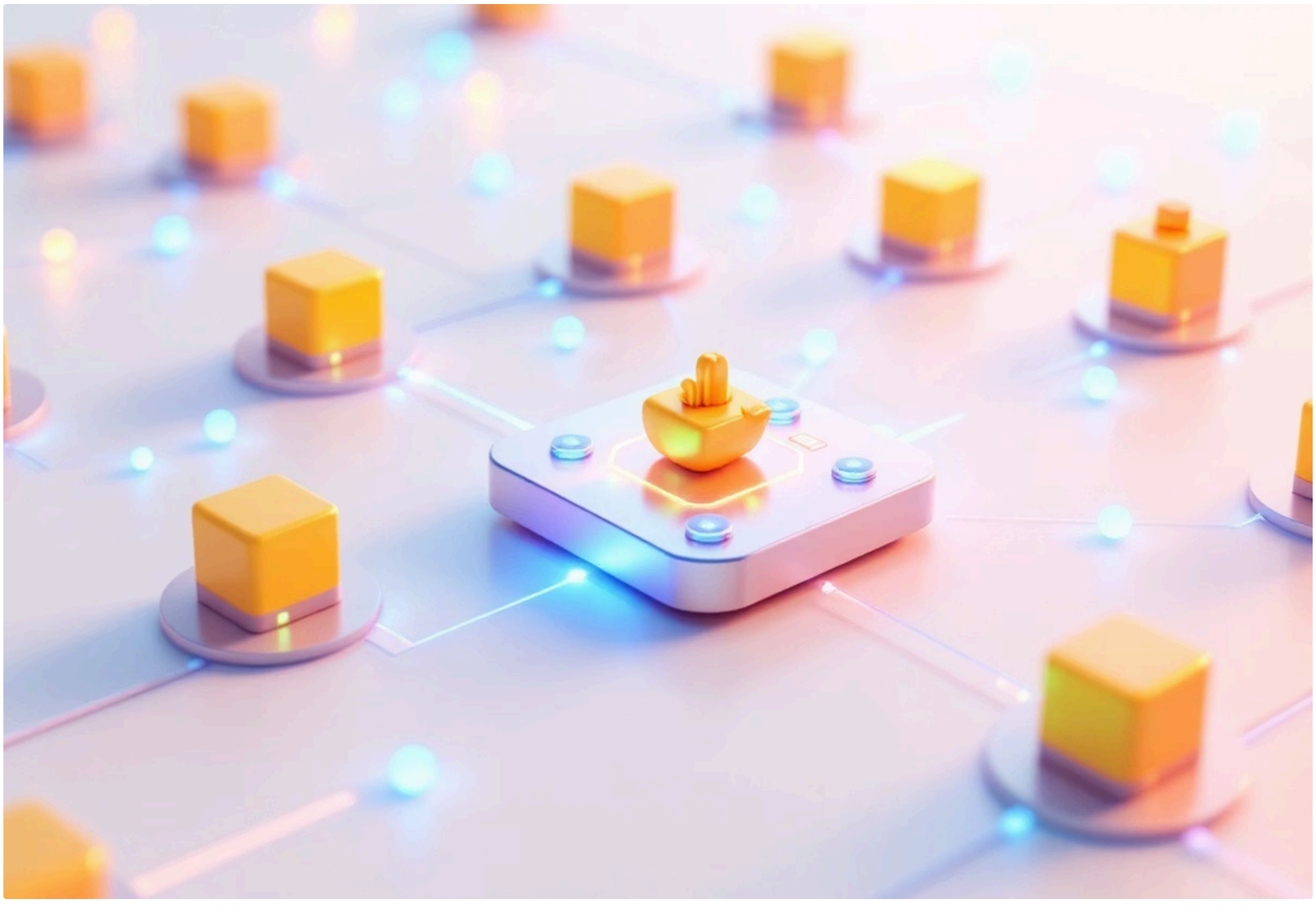
Configuration : Bridge (par défaut de Docker Compose, sans configuration spéciale).

Note Spécifique : Bien que Portainer soit sur son propre réseau, il a la capacité de **gérer** tous les autres réseaux (pia, wordpress, wordpress3) et conteneurs grâce au montage du socket /var/run/docker.sock depuis l'hôte dans son conteneur.

Made with GAMMA

Décomposition d'un docker-compose

Le fichier `docker-compose.yml` est au cœur de la gestion des applications multi-conteneurs avec Docker Compose. Il s'agit d'un fichier de configuration au format YAML qui décrit l'ensemble des services, réseaux et volumes nécessaires au fonctionnement d'une application. Il permet de définir l'architecture de votre application de manière déclarative et de l'orchestrer facilement via une seule commande. Ce document vise à explorer sa structure, ses sections clés et son rôle fondamental dans l'orchestration des conteneurs.



Structure Générale d'un Fichier docker-compose

Un fichier `docker-compose.yml` est organisé de manière logique pour définir les composants de votre application. Il utilise la syntaxe YAML, qui est basée sur l'indentation pour structurer les données. Les sections principales sont généralement situées à la racine du document et contiennent des définitions détaillées pour les services, les volumes et les réseaux.

```
version: '3.8'
services:
  web:
    # Configuration du service web
  db:
    # Configuration du service de base de données
volumes:
  db_data:
networks:
  app_network:
```

Made with GAMMA

Sections Principales et Leur Rôle

Chaque section du fichier `docker-compose.yml` joue un rôle spécifique dans la description de l'infrastructure de votre application. Voici un aperçu des composants essentiels :

1

Section `version`

Cette section spécifie la version du format de fichier Docker Compose. Il est crucial de la définir car différentes versions supportent différentes fonctionnalités et syntaxes. La version '3.x' est la plus courante et recommandée pour les nouvelles applications.

2

Section `services`

C'est le cœur du fichier, où vous définissez chaque conteneur qui compose votre application. Chaque clé sous `services` représente un service (ex: `web`, `db`, `backend`), et ses propriétés décrivent comment le conteneur doit être construit et exécuté (image, ports, volumes, réseaux, variables d'environnement, dépendances).

3

Section `volumes`

Cette section déclare les volumes persistants qui seront utilisés par les services. Les volumes sont essentiels pour stocker les données importantes de manière persistante, même si les conteneurs sont supprimés ou recréés. Ils peuvent être nommés ou liés à un chemin sur l'hôte (bind mounts).

4

Section `networks`

Permet de définir des réseaux personnalisés pour les services de votre application. L'utilisation de réseaux isolés améliore la sécurité et la clarté de la communication entre les services, en les séparant du réseau bridge par défaut de Docker.

Exemple de Configuration Détaillée des Services

La section `services` est la plus riche en options. Chaque service peut avoir des configurations spécifiques :

- `image`: L'image Docker à utiliser pour le conteneur (ex: `nginx:latest`, `postgres:13`).
- `ports`: Mappe les ports du conteneur aux ports de la machine hôte (ex: `"80:80"`).
- `volumes`: Monte des volumes persistants ou des fichiers de l'hôte dans le conteneur (ex: `./app:/var/www/html`).
- `environment`: Définit des variables d'environnement spécifiques au service (ex: `MYSQL_ROOT_PASSWORD=secret`).
- `depends_on`: Spécifie les dépendances entre les services (ex: le service `web` dépend du service `db`). Cela assure un ordre de démarrage.
- `networks`: Attache le service à un ou plusieurs réseaux définis.

Bien que la section `environment` puisse exister au niveau racine pour les fichiers d'environnement `env_file`), les variables d'environnement sont le plus souvent définies directement sous chaque service pour une configuration plus granulée.

Made with GAMMA

Orchestration des Conteneurs avec Docker Compose

L'orchestration est le processus par lequel Docker Compose gère le cycle de vie de votre application multi-conteneurs. Lorsque vous exécutez une commande comme `docker-compose up`, Docker Compose lit le fichier `docker-compose.yml` et effectue les étapes suivantes :

Lecture du Fichier

Analyse du `docker-compose.yml` pour comprendre l'architecture souhaitée.

Création des Ressources

Création des réseaux et volumes si ce n'est pas déjà fait.

Construction/Pull des Images

Construction des images (si un `Dockerfile` est spécifié) ou téléchargement des images depuis un registre.

Démarrage des Services

Lancement des conteneurs dans l'ordre défini par les dépendances (`depends_on`).

Configuration des Communications

Connexion des services aux réseaux appropriés et configuration des résolutions DNS internes.

Ce processus garantit que tous les composants de votre application sont correctement configurés et connectés, prêts à fonctionner comme une seule unité cohérente.

Exemple Complet de docker-compose.yml

Cet exemple montre un fichier `docker-compose.yml` pour un déploiement d'une base de donnée MySQL et d'une interface web de gestion de BDD "phpMyAdmin".

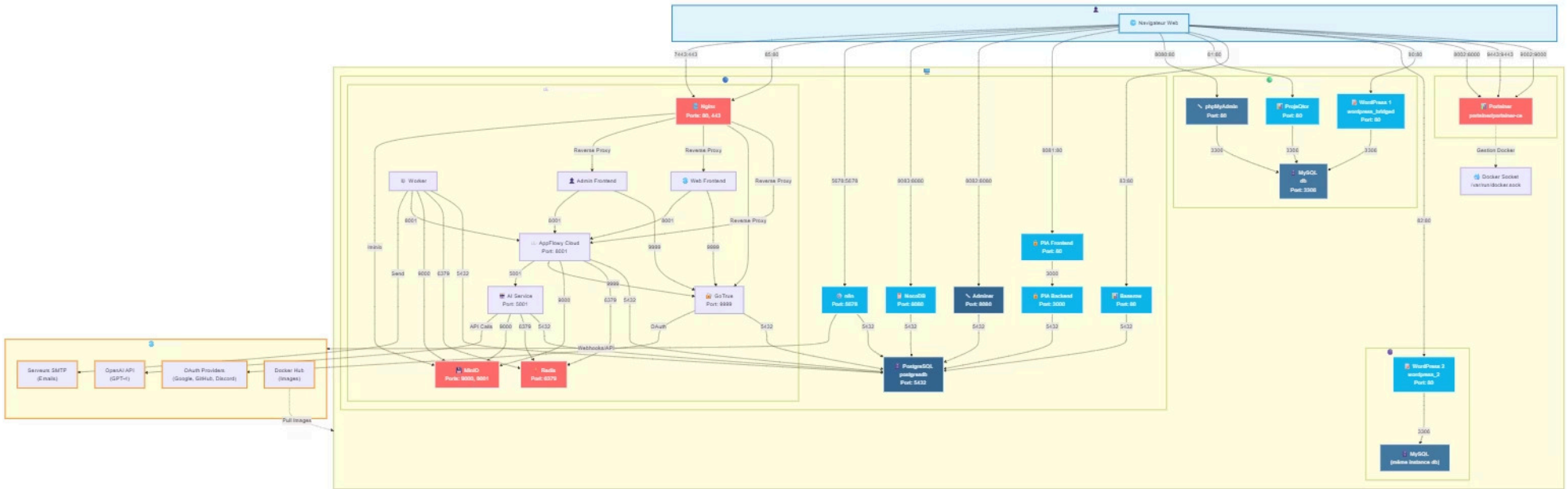
```
services:
  db:
    image: mysql:latest
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wdp1
      MYSQL_USER: wdpuser
      MYSQL_PASSWORD: wdppassword
    networks:
      - frontend
      - frontend3
    volumes:
      - mysql:/var/lib/mysql
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    restart: always
    ports:
      - "8080:80"
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: rootpassword
    networks:
      - frontend
volumes:
  mysql: {}
networks:
  frontend:
    name: wordpress # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
  frontend3:
    name: wordpress3 # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Made with GAMMA

Nom de variable	*Description*
db	Nom du service/container
mysql:latest	Image utilisée
restart	Utilisée pour demandé à docker de lancer les conteneur au démarrage
environment: MYSQL_ROOT_PASSWORD: rootpassword MYSQL_DATABASE: wdp1 MYSQL_USER: wdpuser MYSQL_PASSWORD: wdppassword	Variable d'environnement, elles servent à la configuration de la connexion à la base de donnée
networks: - frontend - frontend3	Permet de donner au container des réseaux de communication vers d'autres services
volumes: - mysql:/var/lib/mysql	Déclaration du volume
phpmyadmin	Nom du service/container
phpmyadmin/phpmyadmin	Image utilisée
always	
environment: PMA_HOST: db PMA_USER: root PMA_PASSWORD: rootpassword	Variable d'environnement, elles servent à la configuration de la connexion à la base de donnée
networks: - frontend	Permet de donner au container des réseaux de communication vers d'autres services
volumes: mysql: {}	Permet de créer un volume de stockage des données.
networks: frontend: name: wordpress driver_opts: com.docker.network.bridge.host_binding_ipv4: 127.0.0.1 frontend3: name: wordpress3 driver_opts: com.docker.network.bridge.host_binding_ipv4: 127.0.0.1	Création des différents réseaux nécessaire à la communication pour les autres services connectés. 2 réseaux sont déclarer : -frontend (wordpress) -frontend3 (wordpress3) Les deux réseaux sont créer sur l'interface 127.0.0.1 en mode bridge.

Made with GAMMA

Diagramme Mermaid représentant l’infrastructure



Made with GAMMA